

**Automatización de la configuración de almacenamiento en máquinas virtuales
GNU/Linux desde el host con Oracle VM VirtualBox**

Estudiantes:

Jorge Iván Acosta Aristizábal

[Compañero 2]



Universidad del Quindío
Facultad Ingeniería
Ingeniería de Sistemas y Computación
Computación en la Nube
Armenia, Quindío
Octubre del 2026

Índice

Introducción	4
1. Marco Teórico y Tecnologías	5
1.1. Virtualización y Oracle VM VirtualBox	5
1.2. Guest Additions	6
1.3. Comunicación Host–Guest	6
1.4. Gestión de Almacenamiento en GNU/Linux	7
1.5. Automatización con Bash	8
1.6. Testing de Scripts	9
2. Diseño de la Solución	9
2.1. Escenario de Aplicación	9
2.2. Arquitectura y Comunicación entre Host y Guest	11
2.3. Algoritmo de Automatización (Idempotente)	13
2.4. Interfaz de la Solución	16
2.5. Sintaxis del Archivo Declarativo	18
2.6. Instalación y Sistema de Construcción	21
3. Implementación	22
3.1. Estructura del Código	22
3.2. Script / Programa Principal	24
3.3. Parametrización para Múltiples VMs	31
3.4. Manejo de Errores y Casos Edge	32
3.5. Verificación Estática y Pruebas Unitarias	34
3.6. Evidencias de las Pruebas de Funcionamiento	35
3.6.1. Prueba 1: Configuración Inicial de VM1	35
3.6.2. Prueba 2: Configuración de VM2	36

3.6.3. Prueba 3: Disponibilidad del Almacenamiento	36
3.6.4. Prueba 4: Persistencia Tras Reinicio	37
3.6.5. Prueba 5: Re-ejecución sobre VM Configurada	37
3.6.6. Prueba 6: Condición que Impide la Ejecución Normal	38
3.7. Tiempo de Ejecución	39
4. Conclusiones	40
4.1. Logros Alcanzados	40
4.2. Problemas Encontrados y su Solución	40
4.3. Herramientas de Inteligencia Artificial Utilizadas	41
4.4. Limitaciones Identificadas	42
4.5. Trabajo Futuro	42

En prácticas previas de la asignatura se realizó manualmente la preparación de un disco adicional para ser utilizado como almacenamiento de datos en una máquina virtual GNU/Linux. Dicha actividad implicó ejecutar secuencialmente los procedimientos de particionado, creación del sistema de archivos (ext4/xfs), montaje y configuración del montaje automático mediante `/etc/fstab` para garantizar persistencia (Nemeth et al., 2017). Si bien estos pasos son fundamentales para la administración de almacenamiento, su ejecución manual resulta repetitiva, propensa a errores humanos y no escala cuando una organización debe aprovisionar almacenamiento en múltiples máquinas virtuales.

El presente trabajo aborda la automatización completa de este proceso desde el sistema operativo *host*, aprovechando las capacidades de Oracle VM VirtualBox. La solución permite que el *host* inicie y prepare una *VM*, establezca comunicación con el sistema operativo invitado mediante las Guest Additions y ejecute automáticamente las acciones necesarias para que el nuevo almacenamiento quede disponible sin intervención manual. Los requisitos fundamentales son la ejecución desatendida, sin ingreso de contraseñas, comandos interactivos ni pulsaciones de tecla; la reproducibilidad, con el mismo resultado en cada ejecución; la reutilización, aplicable a múltiples *VMs* con parámetros configurables; y la idempotencia, con re-ejecución segura sobre *VMs* ya configuradas sin pérdida de datos ni operaciones destructivas (Wikipedia Contributors, 2024).

La comunicación *host-VM* se realiza a través de la Guest Control API (`VBoxManage guestcontrol`), mecanismo nativo de VirtualBox que no requiere configuración de red adicional más allá del modo puente (*bridged networking*) para la obtención de la IP de la *VM* (Oracle Corporation, 2024c, 2024e). La automatización se implementa en Bash para *host* Linux/macOS, siguiendo prácticas robustas de scripting (GreyCat, 2024; Holen, 2024) y validada con el framework de testing BATS (Bats Contributors, 2024). De este modo, el aprovisionamiento deja de depender de pasos manuales y se convierte en un procedimiento desatendido y verificable, cuyos resultados son consistentes entre ejecuciones y cuya repetición no compromete al sistema invitado.

1. Marco Teórico y Tecnologías

La solución descrita en este documento se inscribe en la confluencia de tres áreas complementarias de la administración de sistemas. La virtualización permite ejecutar sistemas operativos completos como *VM* aisladas sobre un mismo equipo físico y expone los recursos, como discos e interfaces de red, que el *host* puede administrar de forma programática (Oracle Corporation, 2024d, 2024e). La administración de almacenamiento en GNU/Linux, por su parte, define el procedimiento que se busca automatizar: dejar un disco disponible y persistente en el sistema invitado (Nemeth et al., 2017). Por último, la automatización mediante scripts convierte ese procedimiento manual en una operación programada, reproducible e idempotente, es decir, capaz de repetirse sin alterar el resultado ni comprometer los datos ya existentes (Wikipedia Contributors, 2024). Estos tres frentes convergen cuando una organización debe preparar almacenamiento en múltiples *VM* sin intervención manual, que es precisamente el escenario abordado a continuación.

1.1. Virtualización y Oracle VM VirtualBox

Oracle VM VirtualBox es un hipervisor de tipo 2 (alojado, *hosted*), es decir, una aplicación que se instala sobre el sistema operativo del *host* y permite ejecutar múltiples sistemas invitados como *VM* independientes sin requerir modificaciones al sistema anfitrión (Oracle Corporation, 2024d). Este modelo resulta apropiado para entornos de trabajo y educativos, donde la instalación sencilla y la gestión centralizada desde una única interfaz son requisitos frecuentes.

Entre las capacidades de VirtualBox relevantes para este trabajo se encuentran la gestión de discos virtuales en formatos como `.vdi` y `.vmdk`, que permiten adjuntar almacenamiento adicional a una *VM* sin intervención sobre hardware físico, y la configuración de red en modo puente (*bridged networking*), en la cual la interfaz de la *VM* se asocia a una interfaz física del *host* y obtiene una dirección IP propia dentro de la red local

(Oracle Corporation, 2024e). Asimismo, VirtualBox expone una interfaz completa de línea de comandos, `VBoxManage`, mediante la cual es posible crear, iniciar, detener y consultar *VM*, administrar discos y snapshots y consultar o modificar propiedades; dicho comando constituye el punto de entrada para toda la automatización descrita en este documento (Oracle Corporation, 2024d).

1.2. Guest Additions

Las Guest Additions son un conjunto de controladores de dispositivo y servicios que se instalan dentro del sistema operativo invitado y que mejoran la integración y el rendimiento entre el *host* y la *VM*: gráficos acelerados, portapapeles compartido, redimensionamiento dinámico de la ventana y sincronización de hora, entre otros (Oracle Corporation, 2024b).

Su componente de mayor relevancia para este trabajo es el servicio de Guest Control, que habilita desde el *host* el comando `VBoxManage guestcontrol`. A través de este mecanismo es posible ejecutar programas en el invitado, copiar archivos, crear directorios y gestionar sesiones, autenticándose con un usuario y contraseña del propio sistema invitado (Oracle Corporation, 2024a, 2024c). Para que todo ello funcione, las Guest Additions deben estar instaladas en la *VM* y el servicio `VBoxService` debe encontrarse en ejecución; de ello depende la ejecución desatendida que la solución propone (Oracle Corporation, 2024b).

1.3. Comunicación Host–Guest

Existen varios mecanismos para ejecutar comandos en el invitado desde el *host*, cada uno con un distinto nivel de dependencia respecto de la configuración de red.

La Guest Control API (`VBoxManage guestcontrol`) permite ejecutar procesos, transferir archivos y crear directorios dentro de la *VM* sin que el invitado exponga servicios de red adicionales, pues la comunicación la transporta el propio hipervisor por medio de las Guest

Additions (Oracle Corporation, 2024a, 2024c). Como alternativa, SSH con autenticación por clave ofrece un canal estandarizado y ampliamente disponible en sistemas GNU/Linux, siempre que la VM ejecute un servidor SSH y exista conectividad de red; su configuración comprende el intercambio de llaves y opciones de cliente como `StrictHostKeyChecking`, `UserKnownHostsFile` y `ConnectTimeout` (OpenSSH Project, 2024a, 2024b). Finalmente, las carpetas compartidas facilitan la transferencia de archivos entre *host* y *VM*, pero no sustituyen la ejecución remota de comandos (Oracle Corporation, 2024a).

La solución adopta la Guest Control API por ser nativa de VirtualBox y por reducir la superficie de configuración: no exige instalar ni habilitar servicios de red en el invitado ni gestionar llaves SSH. El modo puente se configura igualmente para que la VM obtenga una dirección IP propia en la red local, lo que permite identificarla y verificar su estado durante la ejecución de la automatización (Oracle Corporation, 2024c, 2024e).

1.4. Gestión de Almacenamiento en GNU/Linux

En GNU/Linux el almacenamiento se modela mediante dispositivos de bloques, cuyos nombres dependen del controlador: `/dev/sdX` para discos SCSI, SATA y USB, `/dev/vdX` para discos virtuales y `/dev/nvme*` para unidades NVMe (The Linux Kernel Archives, 2024). Sobre un disco se crean particiones con herramientas como `fdisk` o `parted`; la tabla puede ser MBR o GPT, y esta última es más moderna y recomendada (Nemeth et al., 2017).

Cada partición requiere un sistema de archivos, por ejemplo `ext4` o `xfs`, que se crea con la utilidad correspondiente (`mkfs.ext4`, `mkfs.xfs`) (Nemeth et al., 2017). Dicho sistema de archivos se hace accesible en un punto de montaje de la jerarquía de directorios, para la cual el estándar de jerarquía del sistema de archivos reserva rutas como `/mnt` y `/media` (Linux Foundation, 2015). La operación `mount` asocia la partición con su punto de montaje, pero ese vínculo no persiste tras un reinicio salvo que se declare en `/etc/fstab` o en una unidad de montaje de `systemd` (Nemeth et al., 2017; systemd Developers, 2024).

Para que la identificación sea estable, las entradas de `/etc/fstab` deben referenciar

el sistema de archivos mediante su UUID, obtenido con `blkid`, y no mediante `/dev/sdX`, ya que la letra asignada a un dispositivo puede cambiar entre reinicios o al conectar otro disco (Ubuntu Community, 2023). Una vez aplicada la configuración, su verificación se realiza con herramientas como `lsblk`, `findmnt` y `df -h` (Nemeth et al., 2017).

1.5. Automatización con Bash

La automatización de la preparación de almacenamiento exige un lenguaje disponible de forma nativa en el *host*, con acceso directo a las utilidades de línea de comandos de GNU/Linux y con la madurez necesaria para construir scripts robustos. La solución se implementa íntegramente en Bash, intérprete de comandos presente de forma predeterminada en los sistemas GNU/Linux y macOS y heredero del lenguaje de comandos definido por la norma POSIX, lo que favorece su portabilidad dentro de ese entorno (GreyCat, 2024; IEEE and The Open Group, 2018). Con ello, el alcance queda concentrado en una única implementación en Bash, en lugar de mantener además una versión paralela para otros sistemas operativos.

La robustez de los scripts se apoya en prácticas consolidadas del lenguaje: activar el modo estricto de ejecución (`set -euo pipefail`), entrecomillar adecuadamente las expansiones, emplear arrays y funciones en lugar de cadenas de comandos y verificar explícitamente los códigos de salida de cada instrucción (GreyCat, 2024). Sobre esa base, el algoritmo incorpora el principio de idempotencia: una operación idempotente produce el mismo resultado sin efectos secundarios aunque se aplique repetidamente, de modo que re-ejecutar la automatización sobre una VM ya configurada no reformatea discos ni duplica entradas en `/etc/fstab` (Hochstein & Zadka, 2023; Wikipedia Contributors, 2024).

La calidad de ese código se verifica con dos herramientas complementarias. ShellCheck es un analizador estático que examina los scripts antes de ejecutarlos y detecta errores comunes y malas prácticas, como expansiones sin entrecomillar, comandos inexistentes o manejo deficiente de la salida, sin necesidad de correr el programa (Holen, 2024). BATS

(*Bash Automated Testing System*), en cambio, ejecuta pruebas automatizadas sobre el comportamiento del script, incluida la re-ejecución idempotente sobre una *VM* ya preparada (Bats Contributors, 2024). De este modo, los defectos se detectan en etapa de desarrollo y no durante la configuración del sistema invitado.

1.6. Testing de Scripts

La verificación de los scripts combina análisis estático y pruebas funcionales. El análisis estático se realiza con ShellCheck sobre el código fuente, lo que permite identificar defectos potenciales en etapa de desarrollo y reduce el costo de corrección respecto a los fallos detectados durante la ejecución sobre una *VM* (Holen, 2024). Dicha verificación se ejecuta de forma previa a cualquier prueba funcional, de manera que solo los scripts que superan el linter avanzan a la siguiente etapa (GreyCat, 2024).

Las pruebas funcionales se implementan con BATS, framework diseñado específicamente para scripts Bash, que organiza cada caso mediante bloques de configuración y limpieza (*setup/teardown*) y aserciones, y que produce salida en formato TAP, compatible con herramientas de integración continua (Bats Contributors, 2024). En este trabajo, BATS se emplea para verificar los comportamientos críticos de la automatización: la preparación correcta de un disco nuevo, la detección de una configuración existente y, en particular, la re-ejecución sobre una *VM* ya preparada, la cual debe finalizar con éxito sin modificar los datos persistidos (Wikipedia Contributors, 2024).

2. Diseño de la Solución

2.1. Escenario de Aplicación

El escenario de trabajo lo integra un equipo *host* que ejecuta GNU/Linux con Oracle VM VirtualBox 7.2 y con la solución en Bash instalada, junto con dos máquinas virtuales

GNU/Linux llamadas VM1 y VM2, que comparten el mismo sistema invitado, Debian 13 (trixie), pero se configuran con parámetros distintos. El *host* participa en la red local a través de una de sus interfaces físicas. En este escenario esa interfaz es `enp2s0`, aunque el nombre depende de la topología de cada equipo. Cada VM posee una interfaz de red en modo puente que la conecta a la red local `192.168.1.0/24` y que en el invitado se denota `enp0s3`. Tal designación resulta del controlador y del orden de los adaptadores, de modo que, igual que las direcciones, constituye un valor del escenario y no un dato del que la solución dependa. En el modo puente la dirección tampoco la fija la VM, sino que la ofrece el servidor DHCP de la red local, de modo que no puede predecirse a partir de la configuración del escenario (Oracle Corporation, 2024e). El entorno de pruebas asigna `192.168.1.51` a VM1, `192.168.1.52` a VM2 y `192.168.1.10` al *host*. Ese plan describe únicamente el entorno y no constituye una premisa de la solución, la cual identifica la dirección en tiempo de ejecución, tal como se detalla en la Subsección 2.2. Asimismo, ambas cuentan con un disco virtual adicional (`.vdi`) que la automatización deja montado como almacenamiento de datos: `/mnt/datos` con `ext4` en la primera y `/srv/datos` con `xfs` en la segunda (Nemeth et al., 2017). La comunicación entre el *host* y cada VM se realiza con la Guest Control API, que se apoya en las Guest Additions instaladas en el invitado (Oracle Corporation, 2024c).

La Figura 1 representa el escenario mediante un diagrama de despliegue UML elaborado con PlantUML, en el que se distinguen la interfaz física del *host*, la red local en modo puente, la interfaz de cada VM, los discos de almacenamiento y el flujo general de la automatización en cinco pasos: (1) la solución enciende la VM con `VBoxManage startvm`; (2) identifica en tiempo de ejecución la dirección IP de la VM a partir de la propiedad que publican las Guest Additions, con respaldo sobre la tabla ARP de la red en puente; (3) ejecuta comandos en el invitado con `VBoxManage guestcontrol`; (4) prepara el almacenamiento mediante la secuencia de identificación del disco, particionado, creación del sistema de archivos, punto de montaje, entrada en `/etc/fstab` y montaje; y (5) verifica el resultado

con `findmnt` y `df -h`, incluida la re-ejecución idempotente sobre una VM ya configurada (Nemeth et al., 2017; Oracle Corporation, 2024b; Wikipedia Contributors, 2024). El mismo flujo se aplica sobre VM2 con otros parámetros, lo que evidencia la reutilización de la solución. Las direcciones IP y los nombres de interfaz que aparecen en la figura describen el entorno de pruebas y no son datos de entrada de la solución: la dirección la obtiene ésta por sí misma en el paso 2 y de los nombres de interfaz no hace uso alguno.

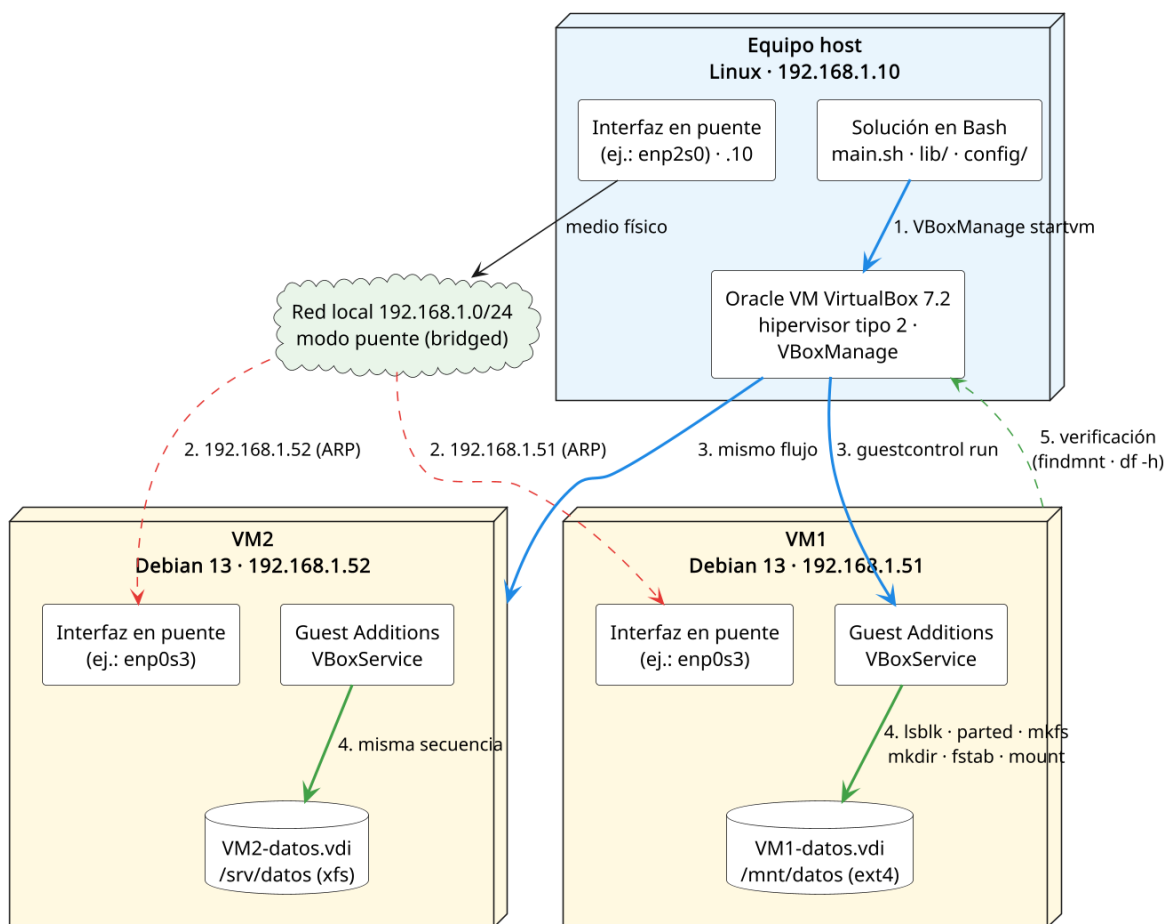


Figura 1: Escenario de despliegue: *host*, VirtualBox, red en modo puente, VM y almacenamiento adicional

2.2. Arquitectura y Comunicación entre Host y Guest

La solución se estructura en cuatro componentes. Un punto de entrada único, `vboxdisk`, recibe una orden (`apply`, `status` o `ld`) junto con el archivo declarativo que describe el

estado deseado y coordina la ejecución; las bibliotecas de `src/lib/` concentran funciones reutilizables de orquestación, comunicación, estado y almacenamiento; el archivo `vdisk.yml` declara, en un bloque por *VM*, los parámetros que cambian entre máquinas: disco objetivo, punto de montaje, sistema de archivos y credenciales; y el fichero de estado `state.lock` registra el último cambio realizado por el sistema sobre cada *VM* y las huellas con las que se detectan modificaciones externas (GreyCat, 2024). Con esta separación, la misma orden se aplica a *VM1* y *VM2* sin modificar el código, y las verificaciones con ShellCheck y BATS descritas en el Marco Teórico recaen sobre funciones pequeñas e independientes (Bats Contributors, 2024; Holen, 2024). El lenguaje de implementación es Bash, justificado en la Subsección 1.5 por su disponibilidad nativa en el *host* y por su acceso directo a la interfaz de línea de comandos `VBoxManage` (Oracle Corporation, 2024d).

La comunicación con el invitado se establece con la Guest Control API (`VBoxManage guestcontrol`), mecanismo que satisface el requerimiento de utilizar las Guest Additions para la interacción (Oracle Corporation, 2024c). En este mecanismo el *host* entrega las órdenes al hipervisor, éste las transporta al servicio `VBoxService` en ejecución dentro de la *VM* y el resultado regresa por el mismo canal; ese trayecto no atraviesa la red en puente, de modo que la ejecución de comandos no depende de conocer de antemano la dirección IP del invitado (Oracle Corporation, 2024a, 2024b). La autenticación se realiza con un usuario y una contraseña del propio sistema invitado, leídos del archivo de configuración antes de abrir la sesión, de manera que ninguna etapa solicita credenciales, comandos ni pulsaciones al usuario (Oracle Corporation, 2024c).

La dirección IP de la *VM* sí se identifica de forma automática, porque es un requisito del proceso y se emplea para verificar la conectividad y registrar la máquina atendida. El mecanismo principal consulta la propiedad que publican las Guest Additions en el hipervisor. La orden `VBoxManage guestproperty get VM1` lee la propiedad `/VirtualBox/GuestInfo/Net/0/V4/IP` y devuelve la dirección vigente, sin sondear la red

ni exigir servicios adicionales en el invitado (Oracle Corporation, 2024b, 2024d). Como respaldo, y en particular si la propiedad aún no se ha actualizado tras el arranque, la solución consulta la tabla ARP completa del *host* con `ip neigh` y selecciona la entrada cuya dirección MAC coincide con la del adaptador de la *VM* reportada por `VBoxManage showvminfo`, operación que no exige conocer el nombre de ninguna interfaz; si éste fuera necesario, se obtiene en tiempo de ejecución con `ip route get` a partir de la dirección recién identificada. Del mismo modo, la solución nunca utiliza el nombre de la interfaz del invitado, el cual ni la Guest Control ni la consulta de propiedades requieren. En ambos casos la dirección IP es un resultado que la solución obtiene, valida y registra, nunca un valor fijo codificado en el programa (Oracle Corporation, 2024e).

2.3. Algoritmo de Automatización (Idempotente)

El algoritmo se organiza en tres etapas que se ejecutan de forma secuencial y, en el camino normal, sin intervención del usuario: una etapa de preparación en el *host*, una etapa de preparación del almacenamiento que se despacha al invitado con `VBoxManage guestcontrol` y una etapa de verificación y cierre que vuelve a ejecutarse en el *host*. La Figura 2 resume ese flujo en un diagrama de procesos con dos carriles, lo que permite distinguir en cada nodo qué equipo ejecuta la acción.

La etapa de preparación comienza con la verificación declarativa: el punto de entrada carga el estado deseado de `vdisk.yml` y lo contrasta con el registro de `state.lock`. Si no hay cambios pendientes y la *VM* permanece apagada, se verifica además que la huella del disco virtual, esto es, la fecha de modificación del fichero `.vdi` y la configuración reportada por `VBoxManage showvminfo`, coincida con la registrada tras la última aplicación; como un sistema invitado apagado no admite escrituras en su interior, una huella idéntica permite omitir la *VM* sin abrir sesión en ella (Wikipedia Contributors, 2024). Si la huella cambió o la *VM* está encendida, la solución realiza un sondeo de solo lectura en el invitado y lo compara con el registro: una discrepancia atribuible a una modificación externa detiene

la operación sobre esa *VM* y solicita confirmación antes de forzar la sincronización con `vdisk.yml`, por el riesgo de pérdida de datos, de manera que la corrección también puede hacerse manualmente. Estas confirmaciones se limitan a los caminos peligrosos, como la deriva detectada o el ciclo de encendido necesario para adjuntar un disco que falta, de modo que la ejecución habitual permanece desatendida (Oracle Corporation, 2024d).

Superada la verificación declarativa, el punto de entrada comprueba si la *VM* está encendida y, de no estarlo, la enciende con `VBoxManage startvm` y aguarda la respuesta de `VBoxService`; después identifica su dirección IP en tiempo de ejecución, según se describió en la Subsección 2.2, y despacha las órdenes al invitado (Oracle Corporation, 2024b, 2024c).

La etapa de almacenamiento es idempotente porque cada acción está precedida por una guarda que consulta el estado actual antes de modificarlo (Wikipedia Contributors, 2024). La primera guarda utiliza `lsblk` para identificar el disco adicional por su tamaño y comprobar si carece de tabla de particiones; solo en ese caso crea una tabla GPT con una partición mediante `parted` (Nemeth et al., 2017). La segunda verifica, con `blkid`, si la partición ya contiene un sistema de archivos; de lo contrario ejecuta `mkfs.ext4` o `mkfs.xfs` según la configuración de la *VM*. La tercera consulta `findmnt` y el contenido de `/etc/fstab` y, únicamente si el montaje no está declarado ni activo, obtiene el UUID, crea el punto de montaje con `mkdir -p`, añade la entrada por UUID para que el montaje persista entre reinicios y ejecuta `mount -a` (Linux Foundation, 2015; Ubuntu Community, 2023).

Finalmente, la etapa de verificación confirma con `findmnt` y `df -h` que el almacenamiento quedó montado y registra el resultado: la ejecución termina con código 0 cuando la verificación es correcta y con uno de los códigos de error de la Subsección 2.4 cuando alguna guarda o verificación falla, de modo que el estado inconsistente se detecta de inmediato y la causa queda distinguible en la salida (GreyCat, 2024). Como consecuencia de las guardas, re-ejecutar la solución sobre una *VM* ya configurada no reformatea el disco ni duplica entradas en `/etc/fstab` y deja intactos los datos persistidos; la misma

secuencia aplicada a VM1 y VM2 cambia únicamente los parámetros de entrada (Hochstein & Zadka, 2023; Wikipedia Contributors, 2024).

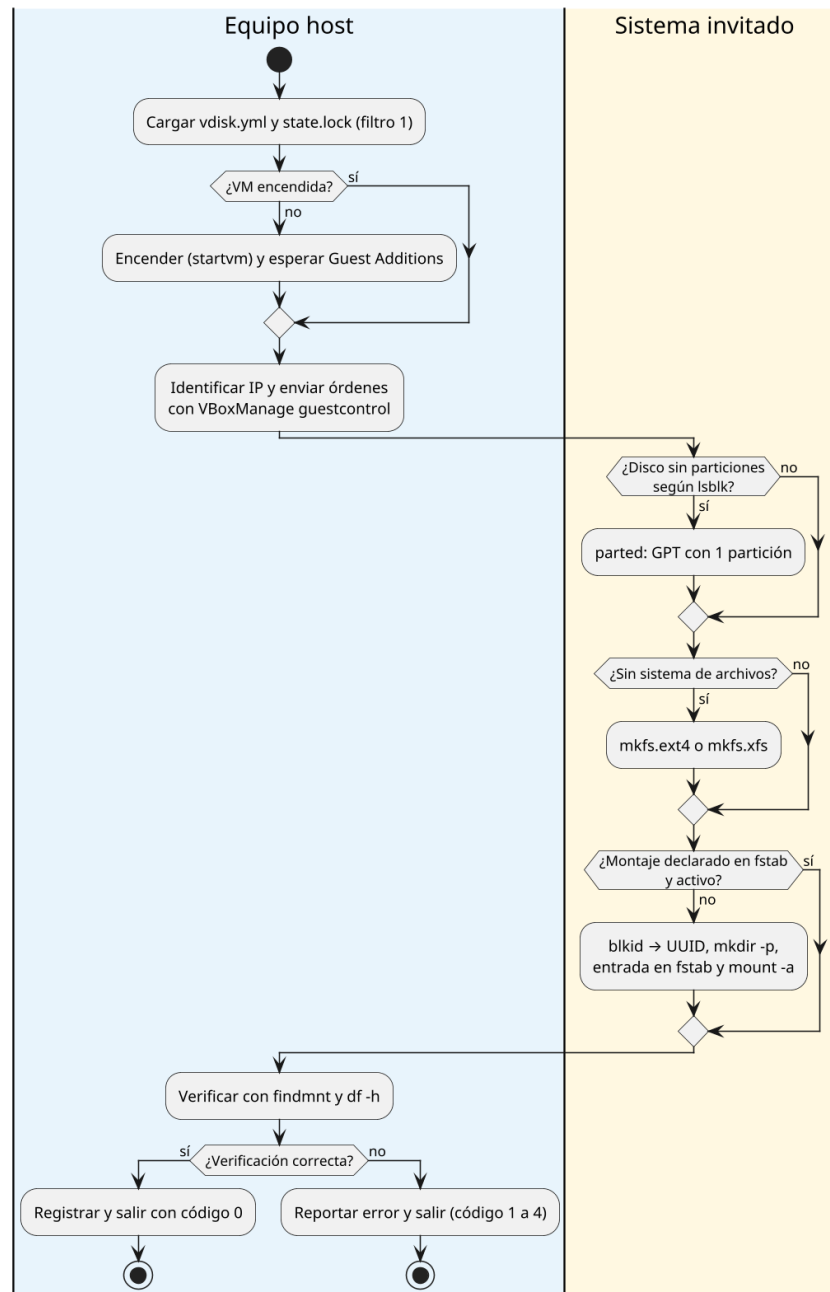


Figura 2: Diagrama de procesos de la automatización idempotente, con las etapas ejecutadas en el *host* y en el sistema invitado

2.4. Interfaz de la Solución

La interfaz de la solución se apoya en un enfoque declarativo inspirado en Terraform: el estado deseado se declara en un archivo y un procedimiento único de convergencia compara lo declarado con la realidad antes de actuar (HashiCorp, 2024). Ese modelo se comparte con Terraform en la separación entre lo declarado y lo real, pero no en el alcance, pues Terraform administra el estado por proyectos, mientras que esta solución trabaja a nivel global, con un único fichero de estado, `state.lock`, que comparten todas las *VM* descritas en el archivo declarativo, dado que el escenario corresponde a un solo conjunto de máquinas. Con el modelo de estado definido, la interfaz propiamente dicha se organiza en tres órdenes, de las que solo una modifica el sistema, como resume la Tabla 1.

Cuadro 1: Órdenes de la interfaz de línea de comandos

Orden	Descripción
<code>vboxdisk apply</code>	Converge todas las <i>VM</i> declaradas en <code>./vdisk.yml</code> hacia el estado deseado. Es la única orden que modifica el sistema y busca el archivo con su nombre estándar en el directorio actual; para usar otro archivo se indica con <code>-f</code> .
<code>vboxdisk apply -f archivo</code>	Ejecuta <code>apply</code> sobre el archivo indicado en lugar de <code>./vdisk.yml</code> .
<code>vboxdisk apply --dry-run</code>	Muestra el plan de cambios resultante de la verificación declarativa, sin modificar nada.
<code>vboxdisk apply -y</code>	Omite las confirmaciones de los caminos peligrosos para la ejecución no interactiva.
<code>vboxdisk status</code>	Lista las <i>VM</i> del archivo con su estado de sincronización y su dirección IP actual si están encendidas, o la última registrada, con la nota de apagada, en caso contrario.
<code>vboxdisk ld nombre</code>	Muestra la tabla de particiones de la <i>VM</i> indicada tal como quedó registrada en <code>state.lock</code> , con la fecha del registro. Solo lee y nunca enciende la <i>VM</i> .
<code>vboxdisk --help</code>	Muestra la ayuda de la solución y termina.
<code>vboxdisk --version</code>	Muestra la versión instalada y termina.

Durante `apply` la solución informa del avance mediante una barra de progreso por etapa y dos cronómetros: el tiempo hasta que la *VM* queda disponible para recibir las órdenes de configuración, es decir, hasta que `VBoxService` responde, y el tiempo total de la ejecución, que se reporta en el resumen final. La barra, los registros y las advertencias se emiten por `stderr`, mientras que los datos, como la tabla de `status` o la salida de `ld`, salen por `stdout`, de modo que la salida de datos puede filtrarse con otros programas sin que el progreso la contamine. Cuando la salida no es un terminal, por ejemplo al redirigirla a un fichero para conservar la evidencia de una corrida, la barra se sustituye por una línea plana por etapa, de manera que el registro no contiene secuencias de control y queda listo para analizar los tiempos. En paralelo, la solución escribe esos mismos mensajes de avance y

advertencia en una bitácora por ejecución, ubicada en `$XDG_DATA_HOME/vboxdisk/state/<datetime>.log`, que conserva siempre el formato plano con independencia de si la salida es un terminal; de este modo cada corrida deja una evidencia permanente con su marca de tiempo, sin que el usuario tenga que redirigir la salida manualmente.

La solución distingue los siguientes códigos de salida:

Cuadro 2: Códigos de salida de la solución

Código	Significado
0	Todas las <i>VM</i> convergieron correctamente o no hubo cambios que aplicar.
1	Error de uso o de configuración: orden desconocida, <i>VM</i> declarada que no existe en el archivo o en el hipervisor, o archivo declarativo ausente, inválido o incompleto.
2	Error de comunicación con la <i>VM</i> : <code>VBoxService</code> sin responder o dirección IP no disponible.
3	Error de almacenamiento o de verificación: disco no identificable o sistema de archivos y montaje no disponibles.
4	Operación cancelada por el usuario en un camino peligroso.

Cuando `apply` atiende varias *VM*, el código final refleja el error de mayor severidad: los errores globales, los códigos 1 y 4, prevalecen sobre los particulares y, entre estos, el código 3 precede al 2.

2.5. Sintaxis del Archivo Declarativo

El archivo `vdisk.yml` utiliza YAML, un formato de serialización legible por humanos construido sobre pares `clave: valor` (YAML Project, 2021), y se lee con la herramienta `yq`, declarada como dependencia de ejecución en la Subsección 2.6. La solución emplea únicamente tres reglas de esa sintaxis: las claves de primer nivel son los nombres de las *VM* declaradas, cada *VM* agrupa un bloque de pares `clave: valor` con sangría uniforme, y cualquier línea que comienza con `#` es un comentario. Los valores numéricos se escriben

sin comillas y las cadenas que contienen espacios o caracteres especiales se delimitan con comillas dobles. El siguiente listado muestra el archivo estándar con dos VM que comparten la estructura y difieren en los parámetros de almacenamiento:

Listing 1: Archivo declarativo estándar vdisk.yml

```
1 # vdisk.yml: estado deseado de las maquinas virtuales
2 VM1:
3   vm_user: operador
4   vm_pass: ChangeMe123
5   disk_file: /home/usuario/maquinas/VM1/VM1-datos.vdi
6   disk_size_mb: 4096
7   mount_point: /mnt/datos
8   fs_type: ext4
9   fs_label: datos-vm1
10
11 VM2:
12   vm_user: operador
13   vm_pass_file: /home/usuario/secretos/vm2.pass
14   disk_file: /home/usuario/maquinas/VM2/VM2-datos.vdi
15   disk_size_mb: 4096
16   mount_point: /srv/datos
17   fs_type: xfs
18   fs_label: datos-vm2
```

El bloque de cada VM solo puede declarar las variables reservadas que la solución reconoce, enumeradas en la Tabla 3. Cada variable define un aspecto concreto de la convergencia, y cualquier clave ajena a ese conjunto se rechaza durante la validación.

Cuadro 3: Variables reservadas del archivo declarativo

Variable	Obligatoria	Significado
vm_user	Sí	Usuario del invitado con el que Guest Control abre la sesión.
vm_pass	Sí*	Contraseña de ese usuario, leída como valor literal.
vm_pass_file	Sí*	Ruta alternativa al fichero que contiene la contraseña.
disk_file	Sí	Ruta del disco virtual <code>.vdi</code> en el <i>host</i> . Si no existe, <code>apply</code> lo crea y lo adjunta a la VM.
disk_size_mb	Sí	Tamaño del disco en megabytes, empleado para crearlo e identificarlo en el invitado.
disk_device	No	Dispositivo en el invitado, por ejemplo <code>/dev/sdb</code> . Vacío significa autodetección por tamaño con <code>lsblk</code> .
mount_point	Sí	Punto de montaje en el invitado.
fs_type	Sí	Sistema de archivos, <code>ext4</code> o <code>xfs</code> .
fs_label	No	Etiqueta del sistema de archivos.

Se requiere al menos una de las variables `vm_pass` y `vm_pass_file`.

Al cargar el archivo, la solución valida que cada bloque contenga únicamente las variables de la Tabla 3, que las obligatorias estén presentes y que la VM declarada exista en el hipervisor; cualquier incumplimiento detiene la ejecución con código 1 antes de tocar el sistema. Los valores se resuelven con la siguiente precedencia: las opciones de línea de comandos, las variables de entorno con el prefijo `VBOXDISK_`, el bloque de la VM en `vdisk.yml` y, por último, los valores por defecto de la solución. Por ejemplo, la ruta de montaje de una VM se obtiene con una consulta como `yq -r '.VM1.mount_point'` `vdisk.yml`, lo que mantiene la lógica independiente de la máquina y satisface la reutilización sobre dos VM con parámetros distintos.

La ventaja de este diseño es la extensibilidad: incorporar una VM nueva al proceso consiste únicamente en añadir un bloque con sus variables reservadas en `vdisk.yml`,

sin modificar el código, la interfaz ni las pruebas de la solución, y las mismas órdenes la atienden junto con las ya declaradas. El archivo crece en línea con el número de máquinas, mientras que la lógica permanece invariable, que es precisamente lo que exige la reutilización sobre dos o más VM del enunciado.

2.6. Instalación y Sistema de Construcción

La solución se construye e instala con GNU Make, herramienta estándar de los sistemas GNU/Linux cuyo modelo de objetivos y dependencias permite declarar que la instalación solo procede después de verificar el entorno y pasar las pruebas (Mecklenburg, 2010). La orden `make install` encadena los objetivos `checkdeps`, `lint` y `test`: primero `checkdeps` comprueba en el *host* las dependencias en tiempo de ejecución, que son Bash, VBoxManage, yq e ip, y las de desarrollo, que son GNU Make, ShellCheck y BATS, y lista las que falten sin instalar nada; después `lint` ejecuta ShellCheck sobre el punto de entrada y las bibliotecas y `test` ejecuta las pruebas con BATS; solo si todo resulta satisfactorio se copian los ficheros. Las dependencias del invitado, como las Guest Additions, parted, util-linux y los paquetes de ext4 o xfs, no pueden verificarse desde el *host* y por eso se declaran como requisito de cada VM.

La instalación sigue las rutas de las variables de entorno XDG (Freedesktop.org, 2021) y no escribe fuera del área del usuario, de modo que no requiere privilegios:

Cuadro 4: Rutas de instalación según las variables XDG

Componente	Ruta	Contenido
Ejecutable	<code>~/.local/bin/vboxdisk</code>	Punto de entrada copiado desde <code>src/vboxdisk</code> .
Bibliotecas	<code>\$XDG_DATA_HOME/vboxdisk/lib/</code>	Ficheros <code>src/lib/*.sh</code> .
Recursos	<code>\$XDG_DATA_HOME/vboxdisk/</code>	Plantilla <code>vdisk.yml.example</code> .
Estado	<code>\$XDG_DATA_HOME/vboxdisk/state.lock</code>	Fichero generado por la solución; no se instala ni se elimina.
Bitácoras	<code>\$XDG_DATA_HOME/vboxdisk/state/<datetime>.log</code>	Una bitácora por ejecución de <code>apply</code> , escrita por la propia solución.

Cuando `XDG_DATA_HOME` no está definida equivale a `~/.local/share`, que es su valor por defecto. El ejecutable localiza sus bibliotecas junto al propio fichero cuando corre desde el repositorio y en el directorio de datos cuando corre instalado, de manera que la misma orden sirve para desarrollo y después de `make install`. Además, el objetivo `install` asegura que el directorio `~/.local/bin` esté en la variable `PATH` del perfil del usuario: si la ruta no aparece en `~/.bashrc` o en `~/.zshrc`, la añade dentro de un bloque delimitado que `uninstall` vuelve a retirar, de modo que la orden `vboxdisk` queda disponible en cualquier sesión nueva sin privilegios y sin salir del área del usuario. El objetivo `uninstall` retira el ejecutable, las bibliotecas y ese bloque del `PATH`, y conserva el estado, las bitácoras y el archivo declarativo del usuario.

3. Implementación

3.1. Estructura del Código

La solución implementada se organiza en un punto de entrada único y una biblioteca de funciones que éste carga, de modo que cada responsabilidad queda confinada en un fichero y puede leerse y probarse por separado, tal como recomienda la guía de GreyCat

(2024). La Figura 3 muestra el árbol de ficheros tal como se versiona en el repositorio:

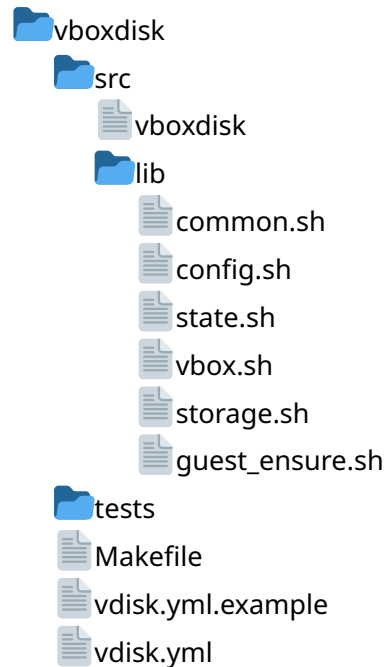


Figura 3: Árbol de ficheros de la solución, con el punto de entrada, las bibliotecas de `src/lib/` y las pruebas de `tests/`

El punto de entrada `src/vboxdisk` coordina la solución: analiza la línea de comandos, localiza y carga las bibliotecas, despacha las tres órdenes descritas en la Subsección 2.4 y orquesta, en el caso de `apply`, las cinco etapas descritas en la Subsección 2.3; la lógica de dominio reside en `src/lib/`. En ese directorio, `common.sh` agrupa los servicios transversales: registro de mensajes con marca de tiempo por `stderr`, los dos cronómetros, la barra de progreso por etapa, las confirmaciones de los caminos peligrosos y la conversión de los códigos de salida de la Tabla 2. `config.sh` lee y valida `vdisk.yml` con `yq` y aplica la precedencia de valores de la Subsección 2.5. `state.sh` administra `state.lock` y las bitácoras por ejecución. `vbox.sh` concentra todas las llamadas a `VBoxManage`: consulta de la configuración de la *VM*, arranque, espera de `VBoxService`, lectura de propiedades, identificación de la dirección IP y despacho al invitado. `storage.sh` calcula las huellas del *host* y el plan de cambios de la verificación declarativa. `guest_ensure.sh` es el único

fichero que se ejecuta dentro de la VM: se copia con Guest Control en cada corrida y aplica las tres guardas de la Subsección 2.3.

El punto de entrada localiza sus bibliotecas en dos lugares, según dónde se ejecute: cuando corre desde el repositorio busca `src/lib` junto al propio fichero y cuando corre instalado busca `$XDG_DATA_HOME/vboxdisk/lib`, en línea con lo que prescribe Freedesktop.org (2021), lo que permite usar la misma orden en desarrollo y después de `make install` sin rutas codificadas. El fichero `vdisk.yml.example` documenta la plantilla declarativa y `Makefile` implementa el sistema de construcción descrito en la Subsección 2.6. El directorio `tests/` contiene las pruebas unitarias de BATS, que emplean un doble de `VBoxManage` en `tests/bin` para ejecutarse sin VM ni privilegios, apoyándose en Holen (2024) y Bats Contributors (2024).

3.2. Script / Programa Principal

El programa se implementó como un punto de entrada único, `src/vboxdisk`, de poco más de cuatrocientas líneas, que carga las bibliotecas de `src/lib/`, registra la trampa que destruye los ficheros temporales de credenciales y analiza la línea de comandos: exige una orden de las tres válidas, acepta las banderas globales `-f`, `--dry-run` y `-y`, y responde a `--help` y `--version` sin requerir una orden. Una orden desconocida o una bandera inválida terminan con el mensaje de uso y el código 1, de modo que la solución nunca adivina lo que el usuario pretendía (GreyCat, 2024). El flujo principal se resume en la Figura 4:

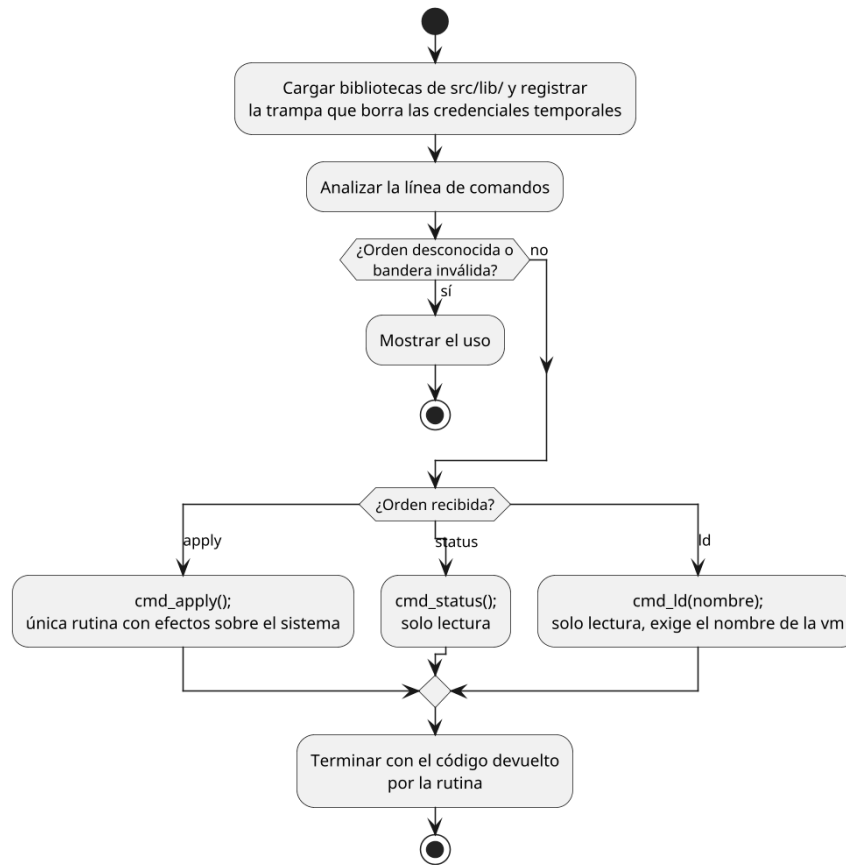


Figura 4: Diagrama de flujo del punto de entrada: análisis de la línea de comandos y despacho de la orden recibida

Las órdenes `status` y `ld` comparten la carga de configuración con `apply`, pero son estrictamente de solo lectura: `status` cruza las entradas de `state.lock` con el estado que reporta `showvminfo` y, cuando la *VM* está encendida, con la propiedad de dirección IP; `ld` muestra la tabla de particiones registrada y la fecha de su registro, sin encender ni abrir la máquina, pues Oracle Corporation (2024d) documenta esas subórdenes como de sola consulta. El despacho en sí es una estructura `case` de pocas líneas:

Listing 2: Despacho de las órdenes en el punto de entrada

```

1 case "$CMD" in
2     apply)
3         cmd_apply
4         ;;

```

```

5      status)
6          cmd_status
7          ;;
8      ld)
9          if [[ -z "$LD_NAME" ]]; then
10             die_cfg "la orden ld requiere el nombre de una vm"
11          fi
12          cmd_ld "$LD_NAME"
13          ;;
14  esac

```

La orden `apply` carga primero la configuración completa: enumera las *VM* declaradas con `yq`, valida cada bloque frente a la Tabla 3, resuelve la precedencia de valores y confirma la existencia de cada *VM* en el hipervisor con una consulta de solo lectura. La validación avanza por etapas: hace que `yq` interprete el fichero y lo rechaza si no es YAML válido; recorre después cada bloque comprobando que el nombre cumpla la forma admitida, que todas sus claves pertenezcan al conjunto reservado, que las obligatorias estén presentes, que exista una credencial declarada y legible y que `fs_type` sea `ext4` o `xf`s. Cualquier incumplimiento se reporta con la *VM* y la clave afectadas y detiene la corrida con código 1, sin haber encendido, modificado ni sondeado máquina alguna.

Superada la validación, `apply` recorre las *VM* declaradas y aplica a cada una la secuencia de cinco etapas de la Subsección 2.3. Cada etapa avanza la barra de progreso por `stderr` y, cuando la salida no es un terminal, emite en su lugar una línea plana con el número de etapa, el nombre y la duración, de modo que la bitácora y las redirecciones contengan marcas de tiempo legibles y no secuencias de control. El cronómetro de disponibilidad se detiene al final de la etapa 2 y el cronómetro total al cerrar la última *VM*, y ambos se reportan en el resumen final (Oracle Corporation, 2024d). La Figura 5 resume la estructura de la corrida por *VM*:

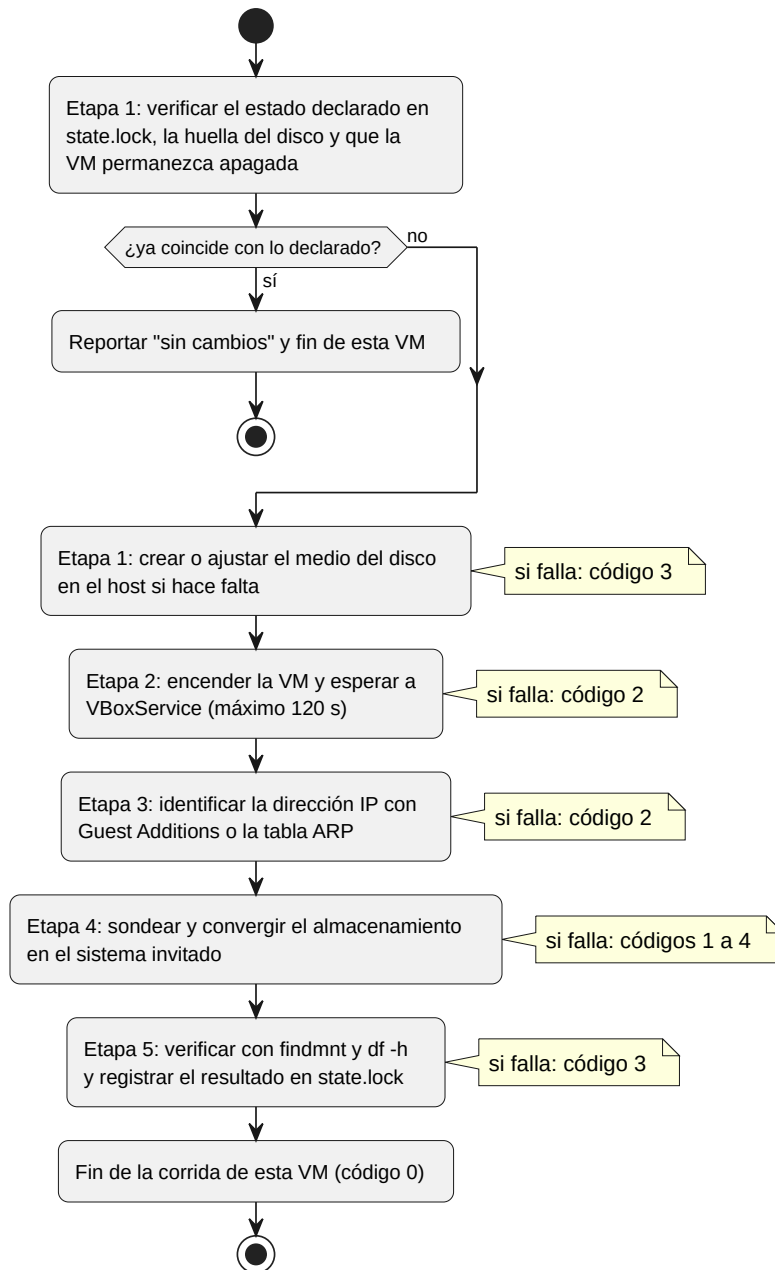


Figura 5: Diagrama de flujo de las cinco etapas de convergencia de `apply` por *VM*, con la decisión temprana de la verificación declarativa y, a la derecha de cada etapa, el código de salida cuando esta falla

La comunicación con el invitado se materializa en una secuencia de llamadas sin interacción. Una crea con `guestcontrol mktemp` un directorio temporal en la *VM*; otras dos copian a ese directorio `guest_ensure.sh` y el fichero temporal de la contraseña con `copyto`; y la última ejecuta el script con `guestcontrol run`, que recibe el usuario y la

contraseña con `--passwordfile`, el directorio de trabajo con `--cwd` y un tiempo máximo con `--timeout`, y captura la salida estándar y de error en el *host* para reenviarla a la bitácora. Como el código que devuelve `run` mezcla los códigos del proceso invitado con los errores propios del servicio Guest Control, el script termina siempre con una línea centinela de la forma `VBOXDISK_EXIT=<n>`, que el *host* interpreta para decidir si la etapa fue satisfactoria, tal como documenta Oracle Corporation (2024c). La copia y la ejecución quedan encapsuladas en dos funciones:

Listing 3: Copia y ejecución del script en el invitado

```
1 vbox_guest_copy_to() {
2     local vm="$1" dir="$2" file="$3"
3     vbox_gc "$vm" copyto --quiet --target-directory="$dir" "$file"
4 }
5
6 # vbox_guest_run <vm> <cwd> <segundos> <script> [args...]:
7 # el programa es el primer argumento tras --, de modo que bash recibe
8 # el script
9 # como operando y sus argumentos tal cual.
10 vbox_guest_run() {
11     local vm="$1" cwd="$2" secs="$3" script="$4"
12     shift 4
13     vbox_gc "$vm" run --timeout="$((secs * 1000))" --cwd="$cwd" -- \
14         /bin/bash "$script" "$@"
15 }
```

La identificación de la dirección IP implementa el mecanismo de la Subsección 2.2 dentro de una función con límite de tiempo: en cada intento consulta la propiedad que publican las Guest Additions y, si aún no está disponible, recurre a la tabla ARP del *host*, emparejando la dirección MAC del adaptador reportada por `showvm` con la normalizada

de `ip neigh`. Ambos intentos se repiten con un intervalo de dos segundos hasta agotar el plazo, y la función devuelve la dirección encontrada o comunica el fallo para que la etapa termine con el código 2 sin llegar al almacenamiento.

El script `guest_ensure.sh` recibe por argumentos los parámetros de la *VM* y opera en dos modos: sondeo, que solo lee y devuelve el estado actual, y convergencia, que aplica las tres guardas. El script se eleva una única vez con `sudo`, leyendo la contraseña del fichero temporal copiado al invitado, y destruye ese fichero al terminar; así ninguna operación pide datos al usuario, en el espíritu de ejecución desatendida que Oracle Corporation (2024c) prevé. La Figura 6 muestra las tres guardas que encadena la función de convergencia, sobre el estado real que arroja el sondeo:

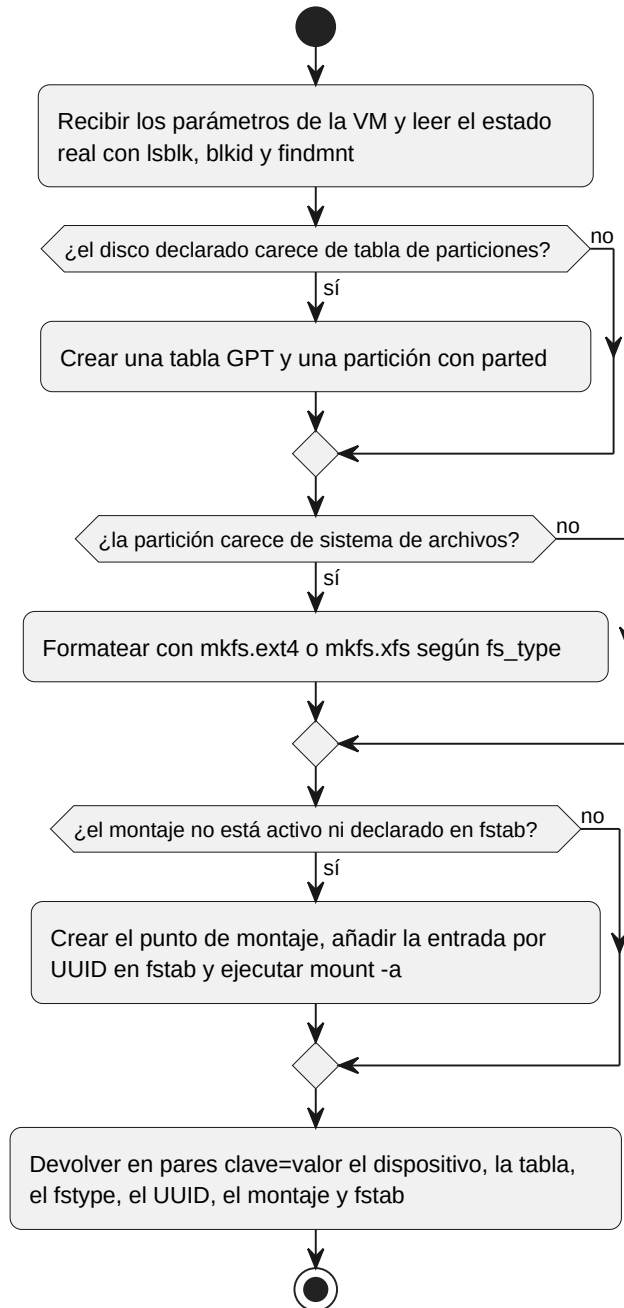


Figura 6: Diagrama de flujo de las tres guardas idempotentes de la convergencia en el invitado: cada pregunta decide si la rama sí aplica la acción o si la rama no la omite y continúa a la siguiente guarda

La primera guarda identifica el disco adicional con `lsblk` por su tamaño declarado y comprueba si carece de tabla de particiones; solo entonces crea una tabla GPT con una partición con `parted`, operación descrita por The Linux Kernel Archives (2024) y tratada en

detalle por Nemeth et al. (2017). La segunda consulta `blkid` y, si la partición no contiene sistema de archivos, ejecuta `mkfs.ext4` o `mkfs.xfs` según `fs_type`. La tercera contrasta `findmnt` y `/etc/fstab` y, únicamente si el montaje no está declarado ni activo, obtiene el UUID, crea el punto de montaje, añade la entrada por UUID siguiendo las recomendaciones de Ubuntu Community (2023) y ejecuta `mount -a` dentro de la jerarquía estandarizada por Linux Foundation (2015). Al finalizar, el script devuelve en pares `clave=valor` el dispositivo, la tabla, el sistema de archivos, el UUID, el estado del montaje y la entrada en `fstab`; con esa salida, el *host* contrasta los valores declarados con la verificación que el propio script ejecuta con `findmnt` y `df -h`, y solo si esa verificación es correcta actualiza `state.lock`, de modo que una ejecución fallida no queda enmascarada en la siguiente corrida.

3.3. Parametrización para Múltiples VMs

El requerimiento de atender más de una *VM* se resuelve separando lo que varía de lo que no varía. El algoritmo de cinco etapas, las órdenes de `VBoxManage`, las tres guardas de almacenamiento, los códigos de salida y el formato de `state.lock` permanecen idénticos para toda máquina; lo único que cambia son los valores del bloque declarativo de la Tabla 3 y la identidad de la *VM* en el hipervisor. Por esa razón ninguna función de `src/lib/` contiene el nombre de una *VM* ni una dirección codificada: el punto de entrada enumera las claves de primer nivel de `vdisk.yml` con `yq eval 'keys'` y, para cada una, lee los parámetros con consultas del tipo `yq -r '.VM1.mount_point'`, en las que solo cambia el nombre de la máquina en curso, tal como permite la convención de claves anidadas que YAML Project (2021) define.

En el escenario descrito, *VM1* y *VM2* comparten el sistema invitado y la estructura del archivo declarativo y difieren en el punto de montaje (`/mnt/datos` frente a `/srv/datos`), en el sistema de archivos (`ext4` frente a `xfs`), en la etiqueta del volumen y en la procedencia de la contraseña, que en un bloque se declara como valor literal y en el otro como ruta de un fichero. El punto de entrada recorre la lista de máquinas y ejecuta la misma función

de convergencia sobre cada una, de modo que incorporar una tercera *VM* consiste en añadir su bloque en `vdisk.yml`, sin modificar el código, la interfaz ni las pruebas, en la misma línea declarativa que HashiCorp (2024) popularizó (GreyCat, 2024). Además, cualquier variable reservada admite sobrescribirse con una variable de entorno con el prefijo `VBOXDISK_`, según la precedencia de la Subsección 2.5, lo que permite ajustar un parámetro puntual en una corrida determinada o en las pruebas automatizadas sin editar el archivo. El estado se almacena en un único `state.lock` con una entrada por *VM*, de manera que la aplicación sobre *VM2* no lee ni escribe el registro de *VM1* (Freedesktop.org, 2021).

3.4. Manejo de Errores y Casos Edge

La solución presupone que cualquier operación puede fallar y que todo fallo debe ser detectable, comunicado con claridad y no destructivo. Cuatro mecanismos sostienen ese objetivo: validación antes de actuar, tiempos de espera con reintentos, confirmación en los caminos peligrosos y códigos de salida diferenciados, descritos en la Subsección 2.4 y resumidos en la Tabla 2. La Tabla 5 detalla la respuesta prevista ante cada situación excepcional.

Cuadro 5: Respuesta de la solución ante situaciones excepcionales

Situación	Respuesta de la solución
Orden o bandera desconocida; <code>vdisk.yml</code> ausente, inválido o con claves ajenas a la Tabla 3	Mensaje de uso o de validación y código 1, sin tocar el sistema.
Credencial ausente o ilegible; VM declarada que no existe en el hipervisor	Rechazo durante la carga de la configuración, código 1.
VBoxManage ausente en el <i>host</i>	Detección en <code>checkdeps</code> al instalar y, en tiempo de ejecución, verificación inicial con código 1.
VM apagada	Se enciende con <code>startvm</code> y se espera a <code>VBoxService</code> reintentando cada dos segundos durante 120 segundos; sin respuesta, mensaje y código 2.
Dirección IP no disponible tras la propiedad de las Guest Additions y el respaldo ARP	Agotados los reintentos, mensaje y código 2, sin iniciar la etapa de almacenamiento.
Disco <code>.vdi</code> ausente en el <i>host</i> o deriva detectada en el sondeo del invitado	Confirmación previa o bandera <code>-y</code> ; sin confirmación, código 4 y VM intacta.
Fallo de <code>parted</code> , <code>mkfs</code> o <code>mount</code> en el invitado	Se detiene la VM en curso, el fallo queda en la bitácora, código 3 y <code>state.lock</code> sin actualizar.
Salida redirigida a fichero o interrupción del proceso	Una línea plana por etapa, sin secuencias de escape; la trampa de salida destruye los ficheros temporales de credenciales.

Los tiempos de espera son finitos y configurables con variables de entorno con el prefijo `VBOXDISK_`: la espera de disponibilidad de la VM y la ejecución del script en el invitado, que se limita además con `--timeout` en `guestcontrol run`. Un tiempo agotado se comunica con su causa en `stderr`, se registra en la bitácora y termina con el código correspondiente, de modo que la solución nunca queda indefinidamente suspendida, en concordancia con las recomendaciones de GreyCat (2024).

El carácter no destructivo descansa en dos reglas. Primera: ninguna operación que destruya datos, como crear una tabla de particiones o ejecutar `mkfs`, se ejecuta sin pasar antes por su guarda de solo lectura, de forma que una VM ya configurada recibe únicamente

las acciones que le faltan, que es precisamente lo que Wikipedia Contributors (2024) define como idempotencia. Segunda: `state.lock` solo se actualiza cuando la verificación final es correcta, de modo que una ejecución interrumpida o fallida deja el registro anterior vigente y la siguiente corrida vuelve a comparar contra la realidad. El modo `--dry-run` materializa este principio sin riesgo: calcula y muestra el plan de cambios de la verificación declarativa en el *host* y no enciende VM alguna ni modifica discos, estado ni archivos. Cuando `apply` atiende varias VM, el código final refleja el error de mayor severidad: los códigos globales 1 y 4 prevalecen sobre los particulares y, entre estos, el código 3 precede al 2; la validación global ocurre antes de cualquier confirmación, de modo que los dos códigos globales no compiten en una misma corrida.

3.5. Verificación Estática y Pruebas Unitarias

Antes de cualquier corrida contra una VM real, la solución se somete a los dos niveles automatizados que encadena `make install`, según se describió en la Subsección 2.6. El primero es estático: ShellCheck analiza el punto de entrada y las bibliotecas y reporta expansiones sin comillas, rutas no protegidas y demás defectos característicos de los programas en shell sin ejecutarlos (Holen, 2024). El segundo son pruebas unitarias con BATS: los ficheros de `tests/` montan en la ruta de búsqueda un doble de `VBoxManage` que responde con salidas fijas, de modo que las funciones se ejercitan sin hipervisor, sin privilegios y en milisegundos (Bats Contributors, 2024). Las pruebas cubren el análisis de argumentos y la conversión de códigos de salida (`cli.bats`), la validación y la precedencia de `vdisk.yml` (`config.bats`) y el cálculo de huellas, la lectura y escritura de `state.lock` y el registro de bitácoras (`state.bats`). El doble permite además reproducir los caminos de error de la Tabla 5, como una VM inexistente o una dirección IP no disponible, así como la interpretación de la salida `clave=valor` y de la centinela del script invitado, sin preparar ninguna condición de fallo en las máquinas reales.

3.6. Evidencias de las Pruebas de Funcionamiento

Las pruebas de funcionamiento verifican la solución contra las VM reales del escenario y corresponden a las seis condiciones exigidas en el punto 5 del enunciado. Cada prueba se describe a continuación en una subsección propia, con su condición inicial, el procedimiento realizado, el resultado esperado y el resultado obtenido, que se completa con cada corrida contra el escenario. La evidencia de cada prueba es doble: la bitácora de la corrida, en la ruta descrita en la Tabla 4, y la captura de pantalla que se muestra al final de cada bloque y que se almacena en el directorio `images/`.

3.6.1. Prueba 1: Configuración Inicial de VM1

Condición inicial. VM1 encendida y `vdisk.yml` declarando su disco adicional de 4096 MB sin tabla de particiones.

Procedimiento. Ejecutar `vboxdisk apply` en el *host* sobre `vdisk.yml` y observar el avance de las cinco etapas hasta el código de salida.

Resultado esperado. El disco queda con una tabla GPT y una partición, formateada en ext4 con la etiqueta declarada, montada en `/mnt/datos` y con su entrada por UUID en `/etc/fstab`; la orden termina con código 0.

Resultado obtenido. [Pendiente]

Evidencia. Bitácora de la corrida, en la ruta de la Tabla 4, y capturas de `lsblk`, `findmnt` y `df -h` dentro de la VM.



Captura pendiente: `images/prueba-1.png`

Figura 7: Prueba 1: configuración inicial del almacenamiento en VM1

3.6.2. Prueba 2: Configuración de VM2

Condición inicial. VM2 encendida y su disco adicional declarado en el mismo archivo `vdisk.yml` sin sistema de archivos.

Procedimiento. Ejecutar la misma orden `vboxdisk apply` sobre el archivo declarativo, de modo que la corrida atienda a las VM declaradas.

Resultado esperado. El mismo flujo de la Prueba 1 aplicando los parámetros de VM2: partición formateada en xfs, montada en `/srv/datos` y con entrada por UUID en `/etc/fstab`; código 0.

Resultado obtenido. [Pendiente]

Evidencia. Bitácora de la corrida y capturas de `findmnt` y `lsblk` dentro de VM2.



Figura 8: Prueba 2: configuración del almacenamiento en VM2 con xfs

3.6.3. Prueba 3: Disponibilidad del Almacenamiento

Condición inicial. Prueba 1 completada en VM1.

Procedimiento. Dentro de VM1, ejecutar `df -h` y `lsblk`, y crear y releer un archivo de prueba en el punto de montaje.

Resultado esperado. `df -h` reporta los 4096 MB del disco declarado y el archivo se escribe y releer sin errores.

Resultado obtenido. [Pendiente]

Evidencia. Captura de `df -h` y del archivo creado en `/mnt/datos`.



Captura pendiente: `images/prueba-3.png`

Figura 9: Prueba 3: capacidad disponible y archivo de prueba en VM1

3.6.4. Prueba 4: Persistencia Tras Reinicio

Condición inicial. VM1 con el almacenamiento configurado según la Prueba 1.

Procedimiento. Reiniciar VM1 y verificar el estado del montaje después del arranque, sin ejecutar la solución.

Resultado esperado. `/mnt/datos` aparece montado automáticamente gracias a la entrada por UUID de `/etc/fstab`, sin ninguna intervención.

Resultado obtenido. [Pendiente]

Evidencia. Capturas de `findmnt` y `df -h` después del reinicio.



Captura pendiente: `images/prueba-4.png`

Figura 10: Prueba 4: montaje automático tras el reinicio de VM1

3.6.5. Prueba 5: Re-ejecución sobre VM Configurada

Condición inicial. VM1 ya configurada, con un archivo persistido en `/mnt/datos`.

Procedimiento. Ejecutar de nuevo `vboxdisk apply` sobre VM1 y comparar el estado antes y después de la corrida.

Resultado esperado. Código 0 sin reformatear el disco, sin duplicar entradas en `fstab` y con el archivo de datos intacto.

Resultado obtenido. [Pendiente]

Evidencia. Bitácora de la segunda corrida y comparación de `blkid` y de `/etc/fstab` antes y después.

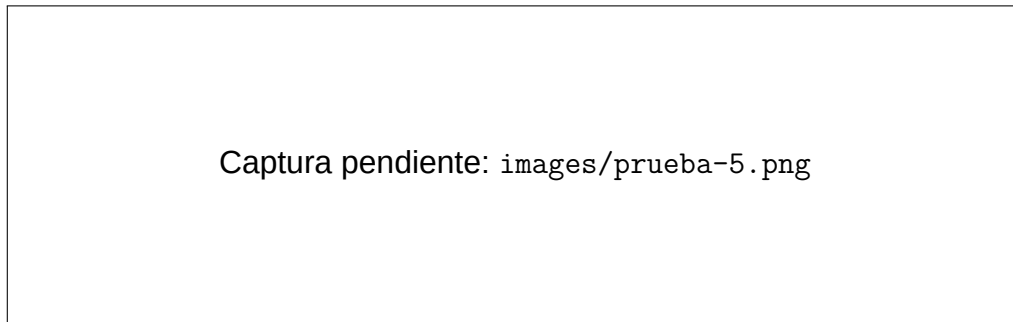


Figura 11: Prueba 5: re-ejecución idempotente sobre VM1 con datos persistidos

3.6.6. Prueba 6: Condición que Impide la Ejecución Normal

Condición inicial. VM1 encendida con el adaptador en modo puente desconectado, de modo que no puede obtener dirección IP.

Procedimiento. Ejecutar `vboxdisk apply` sobre VM1 y agotar los reintentos de identificación de dirección.

Resultado esperado. Mensaje claro de dirección IP no disponible en `stderr`, código 2 y almacenamiento sin modificar.

Resultado obtenido. [Pendiente]

Evidencia. Bitácora de la corrida y captura del mensaje de error.

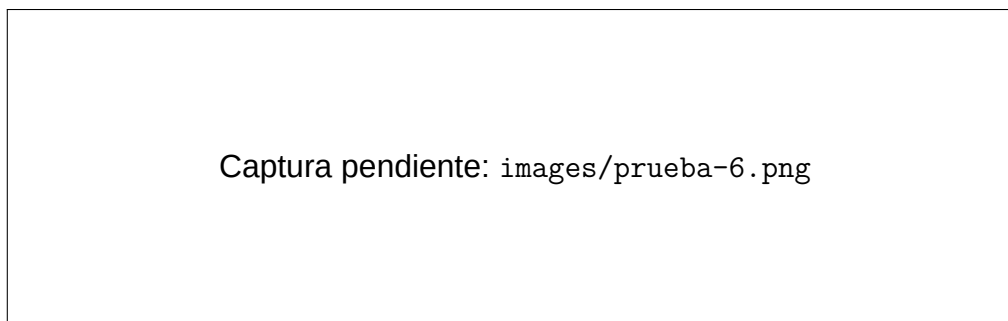


Figura 12: Prueba 6: fallo controlado por dirección IP no disponible

3.7. Tiempo de Ejecución

El tiempo de ejecución se mide con los dos cronómetros que la solución imprime en cada corrida, descritos en la Subsección 2.4: el primero registra el intervalo entre el encendido de la *VM* y el momento en que *VBoxService* responde, es decir, en que la máquina queda disponible para recibir las órdenes de configuración, y el segundo cubre la corrida completa hasta la verificación final. Ambos quedan en la bitácora con la marca de tiempo de cada etapa, por lo que varias corridas pueden compararse sin instrumentación adicional. La Tabla 6 reúne las mediciones de tres corridas consecutivas sobre *VM1*; la columna de condiciones registra el estado inicial de la *VM*, que es la principal fuente de variación, pues una corrida fría incluye el arranque completo del sistema invitado.

Cuadro 6: Mediciones del tiempo hasta la disponibilidad de la *VM* y del tiempo total

Corrida	Condiciones	Disponible (s)	Total (s)	Observaciones
1	VM1 apagada al iniciar	[Pendiente]	[Pendiente]	[Pendiente]
2	VM1 apagada al iniciar	[Pendiente]	[Pendiente]	[Pendiente]
3	VM1 encendida al iniciar	[Pendiente]	[Pendiente]	[Pendiente]

4. Conclusiones

4.1. Logros Alcanzados

Se desarrolló una solución completa de automatización que prepara el almacenamiento adicional de una VM GNU/Linux desde el *host*, sin intervención manual alguna: la solución enciende la VM, espera a que el sistema invitado esté disponible, identifica su dirección IP en tiempo de ejecución y ejecuta la secuencia de identificación del disco, particionado, creación del sistema de archivos, punto de montaje y montaje persistente en `/etc/fstab` (Nemeth et al., 2017; Ubuntu Community, 2023). La comunicación *host*-VM se materializa con la Guest Control API de VirtualBox, de modo que ninguna operación exige contraseñas, comandos interactivos ni pulsaciones de tecla durante la corrida (Oracle Corporation, 2024c).

El resultado es una orden única, `vboxdisk apply`, reproducible porque la misma configuración declarativa conduce al mismo estado final, y reutilizable porque la lógica no está ligada a una VM concreta: el archivo `vdisk.yml` parametriza por máquina el usuario, el disco, el tamaño, el sistema de archivos y el punto de montaje, y la misma corrida convergió sobre VM1 con `ext4` en `/mnt/datos` y sobre VM2 con `xfs` en `/srv/datos`. El algoritmo es idempotente: re-ejecutar la solución sobre una VM ya configurada no reformatea el disco, no pierde los datos persistidos ni duplica entradas en `/etc/fstab`, porque cada acción destructiva está precedida de una guarda de solo lectura que contrasta el estado real con el declarado (Wikipedia Contributors, 2024).

4.2. Problemas Encontrados y su Solución

El desarrollo presentó cuatro problemas que se resolvieron de la siguiente manera.

Código de salida ambiguo

La llamada `VBoxManage guestcontrol run` devuelve códigos que mezclan los del

proceso invitado con los del propio servicio Guest Control, de modo que un fallo del servicio y un código de salida del script no eran distinguibles en el *host*. Se resolvió con una línea centinela `VBOXDISK_EXIT=<n>` emitida por el script invitado sobre su salida estándar: el *host* la interpreta de forma explícita y solo asigna el código 0 cuando la centinela indica convergencia correcta.

Limpieza de credenciales

La trampa de salida del *host* destruía primero los ficheros temporales de contraseña locales, pero el script invitado aún necesitaba el suyo para terminar de eliminarse a sí mismo en el invitado. Se reordenó la secuencia: primero se limpian los directorios temporales del invitado con Guest Control y solo después se revuelven los ficheros del *host*, de modo que ninguna corrida, ni siquiera una interrumpida, deja credenciales en ninguno de los dos lados.

Nombres de VM con guion

Las consultas a `jq` construían las rutas de acceso como `VM1.mount_point`, lo que interpretaba mal los nombres con guion. Se pasan las claves siempre entre comillas (`"${vm}"}.${key}`), de modo que cualquier nombre que supere la validación se lee correctamente.

Restricciones de Guest Control

El servicio no admite la eliminación recursiva de directorios con `guestcontrol rm` y `mktemp` anuncia el directorio creado con un prefijo de texto. Se eliminan los directorios temporales con `guestcontrol run -- /bin/rm -rf` y se extrae la ruta con el prefijo indicado en la documentación de la utilidad (Oracle Corporation, 2024c).

4.3. Herramientas de Inteligencia Artificial Utilizadas

Se empleó un agente de inteligencia artificial basado en un modelo de lenguaje, integrado en la terminal, para las siguientes tareas. En primer lugar, como asistente de

desarrollo: diseño del enfoque declarativo y del algoritmo de cinco etapas, generación del código Bash de las bibliotecas en `src/`, del `Makefile`, de los ficheros de prueba BATS y del doble de `VBoxManage`. En segundo lugar, como verificador documental: se consultó la documentación oficial y el código fuente de VirtualBox para confirmar el comportamiento exacto de `guestcontrol run`, `mktemp`, `copyto` y `rm`, lo que permitió detectar que `rm` no admite eliminación recursiva y corregir el diseño de la limpieza del invitado antes de ejecutarlo. En tercer lugar, como redactor y corrector: redacción de las secciones del presente documento en $\text{\LaTeX}$, seguimiento de las convenciones del trabajo y corrección mecánica de los hallazgos de ShellCheck y de las advertencias de sobreancho de la composición tipográfica. La ejecución de las pruebas contra las VM reales y la validación final de los resultados corresponden a los autores del trabajo.

4.4. Limitaciones Identificadas

La solución exige que las Guest Additions estén instaladas y funcionando en el invitado, pues tanto la espera de disponibilidad como la obtención de la dirección IP dependen de ellas (Oracle Corporation, 2024b). Supone además una red en modo puente accesible desde el *host* y un único disco adicional vacío por VM: no aborda LVM, varios discos ni discos con contenido previo que deba conservarse. Las credenciales de las VM viven en `vdisk.yml` o en un fichero legible por el usuario, suficiente para un entorno de pruebas pero no para uno productivo, donde convendría un almacén de secretos.

4.5. Trabajo Futuro

Extensiones naturales incluyen el soporte de LVM y de varios discos por VM, la ejecución desde un pipeline de integración continua, la traslación de la misma idea declarativa a otras plataformas de virtualización y la sustitución de la autenticación por contraseña por claves de `ssh` cuando el entorno lo permita. En el plano operativo, la inclusión de métricas

y de una bitácora estructurada facilitaría la observabilidad de corridas sobre muchas *VM* simultáneas (Hochstein & Zadka, 2023).

Referencias

- Bats Contributors. (2024). *Bats: Bash Automated Testing System* [Framework de testing para scripts Bash: assertions, setup/teardown, TAP output]. Consultado el 2 de octubre de 2026, desde <https://github.com/bats-core/bats-core>
- Freedesktop.org. (2021). *XDG Base Directory Specification* [Variables de entorno XDG_DATA_HOME, XDG_STATE_HOME y XDG_CONFIG_HOME con sus directorios por defecto]. Consultado el 3 de octubre de 2026, desde <https://specifications.freedesktop.org/basedir-spec/latest/>
- GreyCat. (2024). *Bash Guide: Best Practices* [Guía canónica: quoting, arrays, funciones, manejo de errores, set -euo pipefail]. Bash Hackers Wiki. Consultado el 2 de octubre de 2026, desde <https://mywiki.woledge.org/BashGuide>
- HashiCorp. (2024). *Terraform Documentation* [Enfoque declarativo: archivo de configuración, plan de cambios y orden apply para converger el estado deseado]. HashiCorp. Consultado el 3 de octubre de 2026, desde <https://developer.hashicorp.com/terraform/docs>
- Hochstein, L., & Zadka, M. (2023). *Ansible: Up and Running* (3.^a ed.) [Capítulos sobre idempotencia, módulos declarativos, mejores prácticas de automatización]. O'Reilly Media.
- Holen, V. (2024). *ShellCheck: Shell script analysis tool* [Linter estático para detectar bugs y malas prácticas en scripts Shell]. Consultado el 2 de octubre de 2026, desde <https://www.shellcheck.net/>
- IEEE and The Open Group. (2018). *IEEE Std 1003.1-2017: Shell Command Language* [Especificación POSIX del lenguaje de comandos Shell (sh), base de portabilidad]. The Open Group. Consultado el 2 de octubre de 2026, desde https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html

- Linux Foundation. (2015). *Filesystem Hierarchy Standard (FHS) 3.0* [Estándar de jerarquía de sistemas de archivos: puntos de montaje estándar (/mnt, /media, /opt, /srv)]. Linux Foundation. Consultado el 2 de octubre de 2026, desde <https://refspecs.linuxfoundation.org/fhs.shtml>
- Mecklenburg, R. (2010). *Managing Projects with GNU Make* (3.^a ed.) [Modelo de objetivos y dependencias de GNU Make para verificar, probar e instalar de forma encadenada]. O'Reilly Media.
- Nemeth, E., Snyder, G., Hein, T. R., Whaley, B., & Mackin, D. (2017). *Linux Storage Management* (5.^a ed.) [Capítulos sobre discos, particiones, sistemas de archivos, LVM, RAID, fstab]. Addison-Wesley Professional.
- OpenSSH Project. (2024a). *SSH Key-Based Authentication* [Generación de claves, ssh-agent, ssh-copy-id, authorized_keys, configuración de servidor/cliente]. OpenSSH. Consultado el 2 de octubre de 2026, desde <https://www.openssh.com/manual.html>
- OpenSSH Project. (2024b). *ssh_config: client configuration* [Opciones: StrictHostKeyChecking y ConnectTimeout]. OpenBSD. Consultado el 2 de octubre de 2026, desde https://man.openbsd.org/ssh_config.5
- Oracle Corporation. (2024a). *Advanced Topics* [Incluye: Guest Control API, VBoxManage guestcontrol, autenticación, comunicación host-guest]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/AdvancedTopics.html>
- Oracle Corporation. (2024b). *Guest Additions* [Documentación oficial: instalación, características y uso de Guest Additions]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/guestadditions.html>
- Oracle Corporation. (2024c). *Guest Control: VBoxManage guestcontrol* [Referencia de comandos para ejecutar programas, copiar archivos, crear directorios en el guest]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/AdvancedTopics.html#guestcontrol>

Oracle Corporation. (2024d). *VBoxManage Reference* [Referencia completa de línea de comandos: control de VMs, discos, snapshots, propiedades]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/VBoxManageRef.html>

Oracle Corporation. (2024e). *Virtual Networking* [Modos de red: NAT, Puente (Bridged), Solo host, Red interna]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/Network.html>

systemd Developers. (2024). *systemd.mount: Mount unit configuration* [Unidades de montaje systemd, alternativa a /etc/fstab, dependencias y ordenamiento]. Freedesktop.org. Consultado el 2 de octubre de 2026, desde <https://www.freedesktop.org/software/systemd/man/latest/systemd.mount.html>

The Linux Kernel Archives. (2024). *Block Devices and Partitions* [Documentación del kernel: subsistema de bloques, naming de dispositivos (/dev/sd*, /dev/vd*, /dev/nvme*)]. Linux Kernel Organization. Consultado el 2 de octubre de 2026, desde <https://www.kernel.org/doc/html/latest/block/index.html>

Ubuntu Community. (2023). *Using UUIDs in /etc/fstab* [Por qué usar UUID vs /dev/sdX: persistencia tras reinicios y cambios de hardware]. Ubuntu Documentation. Consultado el 2 de octubre de 2026, desde <https://help.ubuntu.com/community/UsingUUID>

Wikipedia Contributors. (2024). *Idempotence* [Definición matemática y aplicación en DevOps/automatización de sistemas]. Wikipedia. Consultado el 2 de octubre de 2026, desde <https://en.wikipedia.org/wiki/Idempotence>

YAML Project. (2021). *YAML: YAML Ain't Markup Language* [Lenguaje de serialización legible: pares clave valor, mapas con sangría y comentarios]. YAML Project. Consultado el 3 de octubre de 2026, desde <https://yaml.org/>